

Tv Tracker Product Improvement Roadmap

Reliability first / Calm desktop experience / Measured growth

Prepared 2 October 2026 | Current baseline version 2.2.1

Recommended direction

Build a dependable local desktop product around Google Sheets, SQLite and the existing Electron architecture. Strengthen data correctness, recovery, security and release verification first. Then improve responsiveness and simplify the interface. Introduce cloud infrastructure only when a measured requirement justifies it.

The Toyota comparison is a useful product goal: predictable operation, durable recovery and low maintenance. Translate it into testable release gates and performance budgets. This roadmap proposes those gates; it does not claim certification, zero bugs or Toyota-equivalent reliability.

What already works

The current app includes encrypted local settings, offline cache, capture history, sync receipts and retry safeguards, background processing, exports and GitHub release updates. The 2.2.1 QC report records 173 checks and a successful native upgrade. These are foundations to strengthen, not features to rebuild. [1, 2]

Fresh capture remains unverified in the latest recorded live session: login succeeded, but the saved queue did not return usable results. You reported that the portal sometimes goes down. Keep the configured queue unchanged and distinguish upstream unavailability from an empty, healthy queue. Repeat the live capture check when the portal responds. [1]

How to read the ranking

Level	Meaning	Delivery rule
P0	Data safety and release readiness	Required before broad rollout
P1	Daily quality and responsiveness	Next product release priorities
P2	Productivity and maintainability	After the core release gates pass
P3	Conditional future investment	Start only when its adoption trigger is met

Rank 01 is the highest priority. Effort is relative: S = focused change, M = several components, L = significant cross-layer work. Estimates require implementation analysis. UI prototypes can run alongside P0 work; product rollout still depends on P0 verification.

Ranked improvement register

This is the proposed delivery order. Each initiative has implementation details and a verification gate on the following pages.

Rank	Level	Improvement	Effort
01	P0	Sync correctness and field ownership	L
02	P0	Durable jobs and recovery	L
03	P0	Portal resilience and capture completeness	L
04	P0	Credential and desktop security	M
05	P0	Signed releases and safe updates	L
06	P0	Production QA and release gates	L
07	P1	Responsive interface and background work	M
08	P1	Indexed local production cache	L
09	P1	Efficient Google Sheets synchronization	M
10	P1	Playwright automation pilot	M
11	P1	Calm and minimal interface redesign	L
12	P1	Accessibility and Windows usability	M
13	P1	Diagnostics and performance budgets	M
14	P2	Reliable scheduling and notifications	M
15	P2	Order history and focused workflows	M
16	P2	Consistent reports and selective exports	M
17	P2	Maintainability and simpler setup	M
18	P3	Local analytics acceleration	M
19	P3	Team collaboration and roles	L
20	P3	PostgreSQL or Neon shared storage	L
21	P3	Redis for shared workers	M
22	P3	Tauri footprint evaluation	L

Avoid treating every optional technology as a dependency. Redis, Neon, DuckDB and Tauri are decisions to evaluate, not a shopping list for the next release.

Protect data and recover safely

01 P0 Sync correctness and field ownership

Define stable order identifiers, field ownership and normalization for dates, numbers, booleans and blanks. Preserve human-edited Sheet fields. Recover preview numbering from Sheets before the first capture. Make retry operations idempotent, and reconcile confirmed writes through receipts when a response is lost.

Benefit Stops duplicate orders, false failures, numbering drift and accidental overwrites.

Acceptance Repeated sync and retry produce the same result. Existing manual fields remain unchanged. Concurrent changes are detected and surfaced. A readback mismatch never silently marks a job successful.

02 P0 Durable jobs and recovery

Persist capture and sync state transitions in SQLite. Resume or safely reconcile interrupted jobs after process termination, power loss or an update. Stage complete results before making them visible. Add versioned backups, integrity checks and a guided restore drill; retain existing recovery behavior.

Benefit A restart becomes a recoverable interruption instead of an uncertain outcome.

Acceptance Interrupt each job phase and restart. Completed history remains intact, pending work is recoverable, and no incomplete capture appears as complete. Restore a backup on a clean profile and compare counts and identifiers.

03 P0 Portal resilience and capture completeness

Classify login failure, missing page elements, empty results, network timeout and upstream outage separately. Use bounded retries with backoff and jitter, a cooldown after repeated failures, and explicit cancellation. Validate pagination, expected columns and completion evidence before committing a capture. Keep queue 23656 unless configuration evidence warrants a change.

Benefit Handles an unreliable upstream service without creating misleading or partial production data.

Acceptance Test healthy empty queues, multi-page queues, expired sessions, timeouts and interrupted pagination. Failures preserve the previous capture. Complete one fresh live capture when the portal is healthy and verify it against source results.

Data authority

Google Sheets remains the authoritative production store. SQLite owns local jobs, capture history, sync receipts and a rebuildable production replica. Label cached results with their last successful refresh time. Offline browsing and staging can continue, but final cloud confirmation waits for connectivity. Never show a pending write as confirmed.

Secure the desktop release

04 P0 Credential and desktop security

Replace the service-account key previously shared in chat. Keep its replacement in the existing local encrypted credential path, exclude secrets from Git and release archives, and restrict Sheet sharing to intended users. Review Electron sandboxing, context isolation, IPC allowlists and authenticated loopback routes. Validate file paths and avoid leaking cookies, tokens or personal order data in logs.

Benefit Reduces credential exposure and prevents renderer or local API misuse.

Acceptance Repository and release scans find no credentials. Unexpected IPC commands and unauthenticated local requests are rejected. Diagnostic bundles redact secrets. A new local credential connects to the intended Sheet successfully.

05 P0 Signed releases and safe updates

Add Windows code signing and verify installer provenance. Keep the existing user update flow, with release notes, progress and actionable failure states. Publish packaged, tested GitHub Releases from CI: a source push alone is not an installable desktop update. Separate desktop release tags from other repository products. Back up before schema migrations and design compatible recovery paths. [3]

Benefit Users can trust and install updates without losing configuration or history.

Acceptance Verify upgrade from supported prior versions, interrupted download, unavailable release feed, checksum or signature failure, and migration failure. Existing captures and settings survive. Define rollback compatibility before publishing a changed schema.

06 P0 Production QA and release gates

Extend current tests with fault injection and packaged desktop workflows. Cover capture, preview recovery, sync, replay, export, scheduling and updates. Include malformed Sheet data, quota responses, concurrent manual edits, disk-full conditions, corrupted cache and server outages. Run deterministic CI checks plus a controlled live smoke test; publish what was and was not verified.

Benefit Catches real failure modes across the entire data flow rather than checking only the interface.

Acceptance All release-critical workflows pass with zero unexplained loss or duplication. A portal outage produces a safe failed capture, not a fabricated pass. No release is labelled fully verified while the required fresh live capture remains outstanding.

Current release evidence and limits

Version 2.2.1 already corrected numeric and boolean Sheet comparisons, a report-detail crash and slow workbook coloring. Recorded CI startup was about 2 seconds with roughly 388 MB working set; these measurements describe that environment only. The live fresh-capture gap is the next essential verification task. [1]

Make daily work responsive

07 P1 Responsive interface and background work

Profile the renderer, Electron main process and Python engine separately. Move CPU-heavy comparison and export work off the UI path, virtualize large tables, lazy-load heavy screens and charts, and request only needed data. Report progress early and support cancellation. Limit concurrent expensive jobs and release resources when jobs finish. [4]

Benefit The app remains usable while capture, sync or export is running.

Acceptance Meet the proposed navigation and interaction budgets on the reference PC. Scrolling and cancellation remain responsive during the largest representative job. Repeated jobs do not cause sustained memory growth.

08 P1 Indexed local production cache

Expand the existing SQLite layer into a normalized, indexed production replica if profiling shows full JSON loads are limiting search and reports. Use transactions, a single writer strategy and suitable indexes. Evaluate WAL mode for concurrent reads and writes. Cache derived report results with explicit invalidation. Keep cloud authority and local capture authority separate. [5]

Benefit Fast search, filtering and offline browsing without another hosted service.

Acceptance Search the representative dataset within the proposed budget. Cache rebuilding produces identical counts and results. A stale cache is clearly labelled, and replacing it never removes local capture history.

09 P1 Efficient Google Sheets synchronization

Batch compatible reads and writes, avoid repeated full-workbook reads, and retry only appropriate failures. Track quota responses, apply bounded backoff, re-read human-owned fields where needed and reconcile uncertain writes. Google Sheets does not provide a full transactional database model; document remaining multi-user limitations. [6]

Benefit Lower request overhead, fewer quota failures and safer retries.

Acceptance Record request count and duration before and after. Test quota exhaustion, partial batch handling and another editor changing the same order. Never claim a conflict-free write without checking its result.

10 P1 Playwright automation pilot

Pilot Playwright behind the current extractor contract. Prefer semantic locators, page-state assertions and explicit completion signals. Use traces on failures, with local retention and sensitive-data controls. Modern Puppeteer also supports locator waits, so compare reliability and debugging effort before replacing it. Neither tool fixes portal outages. [7, 8]

Benefit Potentially clearer automation, better failure evidence and easier regression testing.

Acceptance Run both extractors against identical healthy fixtures and live queues. Compare records, pagination, timing and failure recovery. Adopt Playwright only if it improves the measured outcome without breaking deployment or increasing operational burden.

Create a calm desktop interface

Yes, an Apple-inspired direction suits Tv Tracker. Borrow the clear hierarchy, restrained styling and sidebar-to-content flow associated with iTunes and Apple desktop apps. Build an original interface with familiar Windows behavior and enough information density for production work. [9, 10]

Proposed navigation

Area	Purpose	Primary action
Overview	Today's work, last successful capture, pending sync and system health	Capture now
Orders	Search, filter, saved views and order details	Open order
Captures	Capture history, preview and comparison	Review changes
Reports	Daily and monthly reports, SLA views and exports	Export selected data
Activity	Job progress, sync receipts, retries and failures	Resolve issue
Settings	Connections, credentials, schedules and updates	Test connection

11 P1 Calm and minimal interface redesign

Use a compact sidebar, a central workspace and an optional details inspector. Keep one obvious primary action per screen. Preserve search and useful dense tables; avoid oversized cards and unnecessary empty space. Keep a small connection and job status area visible, showing the last successful refresh and pending work. Add a guided setup flow for portal credentials, Sheet URL, service account and numbering recovery.

Benefit A quieter interface that makes the next action clear and lowers setup friction.

Acceptance Test capture, finding an order, comparing captures, retrying sync and exporting with five representative users. Target at least 90 percent completion across these tasks without coaching, and record all confusing steps for correction.

12 P1 Accessibility and Windows usability

Use Windows typography and native window controls, dialogs and keyboard conventions. Support system, light and dark themes, visible focus, keyboard navigation and reduced motion. Use text and icons alongside status color. Verify contrast, zoom, screen-reader naming and 100, 150 and 200 percent display scaling. Use WCAG 2.2 as an assessment reference, not an unsupported compliance claim. [10, 11]

Benefit More comfortable use across displays, preferences and accessibility needs.

Acceptance Complete the five core tasks using the keyboard. No clipped controls at supported window sizes or scaling levels. Focus is visible and error/status messages are understandable without color alone.

Visual rules Neutral surfaces, one restrained accent, clear typography, consistent spacing and subtle separators. Use existing accessible icons. Reserve strong color for meaningful states, make animation brief and optional, and give empty, offline, stale and failed states a clear next action.

Improve operations and productivity

13 P1 Diagnostics and performance budgets

Add structured local job logs with correlation IDs, duration, row counts and classified errors. Provide a redacted support bundle and a health screen for portal, Sheets, storage and app version. Keep telemetry opt-in. Track performance trends on representative hardware instead of assuming a framework switch improves speed.

Benefit Faster troubleshooting and evidence for future optimization decisions.

Acceptance A user can export a safe diagnostic bundle. Every failed job has a useful next action or an explicit unresolved category. Performance regressions are detected before release.

14 P2 Reliable scheduling and notifications

Strengthen the existing scheduler for sleep, wake, timezone changes and missed runs. Define whether one missed run is caught up or skipped; prevent overlapping work. Show next run and last result, with quiet notifications only for actionable failures or completed requested work.

Benefit Predictable automation without duplicate jobs or notification fatigue.

Acceptance Test sleep across a scheduled time, restart, clock changes, network outage and an already-running job. Exactly the configured catch-up policy is applied.

15 P2 Order history and focused workflows

Provide an order timeline, saved filters, a Needs attention view, change summaries and consistent drill-down details. Preserve user selections during refresh. Make ownership and timestamps clear so users can distinguish captured data, Sheet edits and pending local work.

Benefit Less repetitive searching and faster resolution of exceptions.

Acceptance A user can find an order, explain its last change and resolve a pending issue without switching repeatedly between screens. Refresh preserves valid context.

16 P2 Consistent reports and selective exports

Use one validated reporting model for UI totals and exports. Support selected date ranges, products and columns, with progress and cancellation. Define stable export schemas for downstream Power BI usage. Make timezone, stale-data indicators and totals consistent across daily, monthly and SLA views.

Benefit Smaller, faster exports and trustworthy reporting.

Acceptance Compare UI totals against exported rows for representative filters. Test empty results, invalid dates and large exports. Export cancellation leaves no misleading final file.

17 P2 Maintainability and simpler setup

Define typed IPC and API contracts, centralize validation and configuration, and separate portal, sync and report adapters. Keep dependencies patched, produce an inventory and document release/support procedures. Add configuration checks, safe reset and concise help within the app.

Benefit Changes are easier to review, onboarding is smoother and support needs less manual investigation.

Acceptance A clean installation can be configured using the guide without developer assistance. Contract changes fail clearly in CI, and a release can be reproduced from its documented source tag.

Choose future technology deliberately

Rank	Option and trigger	Recommendation and tradeoff
18 P3	DuckDB or Polars Local historical analysis becomes a measured bottleneck	Benchmark large aggregations against indexed SQLite first. DuckDB is an in-process analytics engine. Add one tool only when it materially improves workload time; keep jobs and receipts in SQLite. [12]
19 P3	Team collaboration Several users need coordinated shared work	Design an authenticated API, roles, audit history and explicit field ownership. Agree a single data authority, conflict rules and offline synchronization before building shared editing.
20 P3	PostgreSQL or Neon Shared transactions, relationships or concurrent writers exceed Sheets	Use a trusted API between desktop clients and the database; never ship a database administrator credential. Retain local SQLite for offline work. Neon adds cloud/network dependency; scale-to-zero can add resume latency. Plan migration, backups and recurring cost. [13]
21 P3	Redis A server has multiple workers or repeated shared cache demand	Use for shared coordination, caching or rate limiting only with measured need. Set TTLs, memory limits and appropriate persistence. It adds a service to operate and does not replace durable order storage. [14]
22 P3	Tauri Electron still misses a defined memory or installer budget after profiling	Prototype one representative workflow and measure the complete product. An OS WebView can reduce shell footprint, but the Python engine and automation browser still consume resources. A rewrite carries compatibility, IPC, accessibility and updater risk. [15]

Recommended stack for the next release

Keep Electron and React for the desktop interface, the bundled Python engine for processing, SQLite for durable local state and Google Sheets for production data. Pilot Playwright for browser automation. Keep the existing GitHub update channel and improve signing and release gates. No Redis, Neon or Render hosting is required for this stage.

When a cloud edition becomes worthwhile

The trigger is a product requirement: coordinated multi-user editing, centrally managed permissions or cross-device availability. Design that as a separate team capability. A local desktop installation should remain useful during cloud or portal outages, and cloud adoption should include consent, migration validation and a defined operating budget.

Define measurable quality targets

These are proposed acceptance targets, not current results or universal industry standards. Validate them on a recorded Windows 11 reference PC with SSD, 16 GB RAM, current supported Chrome or Edge, and a documented dataset. Include the current production-size dataset and a 100,000-row local search fixture. Report medians and p95 values over repeated runs.

Area	Proposed target	Verification method
Data safety	Zero lost or duplicate committed records across the defined fault matrix	Compare IDs, counts, manual fields and receipts after interruption and replay
Navigation	p95 screen navigation at most 200 ms after initial loading	Measure 50 representative switches with warm local data
User feedback	Visible acknowledgement within 100 ms of a primary action	Measure action-to-feedback while another job is active
Local search	p95 indexed search at most 300 ms on 100,000 rows	Test several common and worst-case supported filters
Startup	Cold start to usable shell at most 5 seconds	Measure 10 starts; report background readiness separately
Resource use	Idle CPU below 1 percent on the reference PC; no sustained memory growth	Record idle after 60 seconds and compare 10 repeated job cycles; use the baseline to set a memory budget
Capture reliability	At least 99 percent successful complete captures across 200 healthy-portal runs	Exclude only independently evidenced upstream failures; report portal availability and app failure rate separately
Updates and restore	All supported upgrade and recovery scenarios pass	Test settings, capture history, schema migration and interrupted install/download
Usability	At least 90 percent core task completion in a five-user study	Observe capture, search, compare, retry and export without coaching

Measure exports against the right workload

Record workbook size, rows, columns, formatting, device and duration. Set the final export budget after profiling that workload. Favor optional columns and selected date ranges before optimizing full-history exports. A fast small export does not prove acceptable performance for the entire production workbook.

Release gate

P0 scenarios pass; metrics meet agreed budgets or have a documented, accepted exception; no unexplained data mismatch remains; signed upgrade and restore work; and a fresh live capture is verified when the portal is healthy. Keep a concise evidence report for every desktop release.

Deliver in controlled stages

Stage	Scope	Exit evidence
A Establish baseline	Reproduce 2.2.1 workflows; profile startup, search, sync and export; complete healthy-portal capture	Recorded dataset, environment, known limitations and repeatable QC fixtures
B Harden core	Ranks 01 to 06; incorporate diagnostic instrumentation from rank 13	Fault recovery, field preservation, signed updates and release gates pass
C Improve experience	Ranks 07 to 13; prototype the new UI while retaining workflow behavior	Performance budgets, extractor comparison and usability/accessibility checks pass
D Add productivity	Ranks 14 to 17 after foundation gates	Scheduling, history, reports and setup work in packaged desktop tests
E Evaluate expansion	Ranks 18 to 22 only when their adoption triggers occur	A benchmark or team requirement proves value; costs and migration risks approved

End to end QC matrix

Configure Validate Sheet ID, service-account access, portal credentials, local permissions and queue URL. Recover existing preview numbering before capture. Test invalid configuration and permission-denied responses.

Capture Test a healthy populated queue, a genuinely empty queue, pagination, invalid rows, duplicates, expired login, upstream timeout and cancellation. Confirm no partial result is promoted to a complete capture.

Preview and sync Compare with the previous capture, preserve manual columns, replay the same job, retry a lost response and change a Sheet field concurrently. Verify receipts and cloud readback rather than relying only on a success notification.

Offline and recovery Disconnect during work, terminate the engine and app, restart, and restore a backup into a clean profile. Confirm cached-data labels and pending-work state. Exercise disk-full and corrupted local-cache paths safely.

Reports and updates Compare UI and export totals; test large exports, empty filters and cancellation. Install a signed upgrade, simulate an interrupted download and check configuration and capture history afterward.

Interface Test keyboard operation, focus, screen-reader names, window resize, display scaling, themes and reduced motion. Verify clear next actions in empty, stale, offline and failure states.

Implementation rule

Make small, reviewable changes behind stable contracts. Preserve working behavior and the desktop branch separation. Run the relevant automated and packaged checks for each change, then do a controlled release review. This document is a proposed plan; no application code has been changed for it.

Evidence and reference material

Repository facts come from the existing architecture and 2.2.1 QC records. Technology recommendations use the official references below, checked for this roadmap. The priority order, UI design and numerical targets are recommendations based on the product goals; the vendors do not guarantee those outcomes.

[1] **Tv Tracker 2.2.1 QC report**

HariKrishna1-DS/powerBI_Sync, docs/TV_TRACKER_2_2_1_QC.md, baseline commit 854fd2b

[2] **Project architecture**

Electron, local engine, Google Sheets, SQLite and offline cache

[3] **Electron code signing**

Packaging and signing guidance

[4] **Electron performance**

Profiling and responsiveness guidance

[5] **Appropriate uses for SQLite**

Desktop and local database use

[6] **Google Sheets batch requests**

Batching API operations

[7] **Playwright actionability and traces**

Locator checks and failure inspection

[8] **Puppeteer page interactions**

Modern locators and automatic waits

[9] **Apple sidebars**

Navigation design reference

[10] **Windows design basics**

Layout and platform guidance

[11] **WCAG 2.2 quick reference**

Accessibility assessment reference

[12] **DuckDB documentation**

In-process analytical database

[13] **Neon compute endpoints**

Compute lifecycle and scale-to-zero behavior

[14] **Redis documentation**

Data structures, caching and persistence

[15] **Tauri architecture**

System WebView and application structure

Additional automation reference: [Playwright Trace Viewer](#). Capture diagnostic traces only when useful, apply retention limits and protect sensitive data.